

Final Report

dcs076

變數設定

```
// input_length
logic [5:0] i_token_length;
// main input matrix
logic [3:0] i_token [0:31][0:7];
logic [3:0] WQ [0:7][0:7];
logic [3:0] WK [0:7][0:7];
logic [3:0] WV [0:7][0:7];
// main operation matrix
logic [10:0] q_matrix[0:31][0:7];
logic [10:0] k_trans_matrix[0:7][0:31];
logic [10:0] v_matrix[0:31][0:7];
logic [24:0] score_matrix [0:31][0:31];
logic [40:0] o_token [0:31];

// threshold
logic [4:0] RAT;
logic [30:0] CAT;

// timer
logic [5:0] input_index;
integer q_index;
integer k_trans_index;
integer v_index;
integer score_index;
integer CAT_index;
integer o_token_index;
integer out_index;
```

首先會先用 `i_token_length` 儲存矩陣的大小，用來計算之後乘法、加法的週期。

矩陣的部分就如助教給的那張圖那般，但不同的是在這裡我的 `k` 矩陣是直接儲存轉置後的結果，所以可以少一個步驟，`o_token` 也是，透過 SLT 的過濾後我們可以得知 `o_token` 有效的只有最後一列，所以矩陣不用大到塞下所有元素的程度。

`RAT`、`CAT` 用來暫存之後要 `threshold` 的平均值。

後面所有的 `index` 都與其命名方式有關，主要是做為一個 `timer` 計算現在的週期哪個元素需要做運算，FSM 該如何動。

FSM 設計

```
typedef enum logic [3:0]
{
    S_IDLE,
    S_INPUT, // input i_token
    S_CALCULATE_Q, // MM q
    S_CALCULATE_K_TRANS,
    S_CALCULATE_V,
    S_CALCULATE_SCORE,
    S_SCORE_CAT,
    S_CALCULATE_O_TOKEN,
    S_OUTPUT
} state_t;
```

我的流程主要是這 9 個流程所控制的，在個別狀態中分別做：

1. **S_IDLE**: 默認的狀態，負責在 output 到下筆測資(下次 i_valid)來之前歸零所有變數，以免在之後運算過程混淆，在此期間也會讀入 i_length 並將其轉成對應的長度，以利之後的讀取及運算。
2. **S_INPUT**: 等到 m_ready 訊號拉起後切換到 input state，根據助教的設定將資料一筆筆傳輸到相對應的位子上
3. **S_CALCULATE_Q**、**S_CALCULATE_K_TRANS**、**S_CALCULATE_V**、**S_CALCULATE_SCORE**、**S_CALCULATE_O_TOKEN**: 在特定的狀態中對特定的矩陣做乘法、加法運算，將運算的複雜度及規模分擔到每個 cycle 當中，透過拉長 latency 的方式減少面積及 critical path。
4. **S_SCORE_CAT**: 透過 combinational 的方式及時運算 column average，隨後進入 cycle 中判別是否要歸零，但因為 score 真正會用到的只有最後一行，因此需要判斷的也是最後一行就好。
5. **S_OUTPUT**: 在這時間內將 score 的最後一行資料從 0 到 i_token_length 一一輸入進 o_data 當中，在最後把狀態調回 S_IDLE，初始化所有數據、變數。

```

always_comb
begin
    next_state = current_state;

    case(current_state)
        S_IDLE: if(m_ready) next_state = S_INPUT;
        S_INPUT: if(input_index == (i_token_length + 24)) next_state = S_CALCULATE_Q;
        S_CALCULATE_Q: if(q_index == (i_token_length * 8)) next_state = S_CALCULATE_K_TRANS;
        S_CALCULATE_K_TRANS: if(k_trans_index == (i_token_length * 8)) next_state = S_CALCULATE_V;
        S_CALCULATE_V: if(v_index == (i_token_length * 8)) next_state = S_CALCULATE_SCORE;
        S_CALCULATE_SCORE: if(score_index == (i_token_length * i_token_length)) next_state = S_SCORE_CAT;
        S_SCORE_CAT: if(CAT_index == i_token_length) next_state = S_CALCULATE_O_TOKEN;
        S_CALCULATE_O_TOKEN: if(o_token_index == 8) next_state = S_OUTPUT;
        S_OUTPUT: if(out_index == 8) next_state = S_IDLE;
        default: next_state = S_IDLE;
    endcase
end

```

▲狀態轉變條件

在這裡可以發現 q、k、v 矩陣的 index 邊界條件都是當 index 等於 i_token_length*8 時才轉換型態，這是因為 q、k、v 矩陣乘法每一個皆是 $(L, D) \times (D, D) = (L, D)$ ，有此可見每個 q、k、v 矩陣皆有 i_token_length * 8 個元素，然而 score 矩陣因為是 $(L, D) \times (D, L) = (L, L)$ ，所以相較於 q、k、v 矩陣來說有 i_token_length * i_token_length 個元素。



矩陣運算

```
always_ff @(posedge clk, negedge rst_n)
begin
    if(!rst_n)
    begin
        q_matrix <= '{default:0};
        q_index <= 'd0;
    end else if(current_state == S_CALCULATE_Q)
    begin
        q_matrix[q_row][q_col] <= (i_token[q_row][0] * WQ[0][q_col])+(i_token[q_row][1] * WQ[1][q_col])+(i_token[q_row][2] * WQ[2][q_col])
        + (i_token[q_row][3] * WQ[3][q_col])+(i_token[q_row][4] * WQ[4][q_col])+(i_token[q_row][5] * WQ[5][q_col])
        + (i_token[q_row][6] * WQ[6][q_col])+(i_token[q_row][7] * WQ[7][q_col]);
        q_index <= q_index + 1;
    end else if(current_state == S_IDLE)
    begin
        q_matrix <= '{default:0};
        q_index <= 'd0;
    end
end
end
```

在 q、k、v、score 矩陣中我在一個 cycle 內開 8 個乘法器和 8 個加法器，原本我的寫法很奇妙，居然用 $q[i][j] <= q[i][j] + \dots$ 的寫法累加八次迴圈的值，但直到後面跑 01 出 bug 才發現這錯誤。

```
else if(i_token_length == 32)
begin
    o_token[o_token_index] <= (score_matrix[i_token_length-1][0] * v_matrix[0][o_token_index])
    + (score_matrix[i_token_length-1][1] * v_matrix[1][o_token_index])
    + (score_matrix[i_token_length-1][2] * v_matrix[2][o_token_index])
    + (score_matrix[i_token_length-1][3] * v_matrix[3][o_token_index])
    + (score_matrix[i_token_length-1][4] * v_matrix[4][o_token_index])
    + (score_matrix[i_token_length-1][5] * v_matrix[5][o_token_index])
    + (score_matrix[i_token_length-1][6] * v_matrix[6][o_token_index])
    + (score_matrix[i_token_length-1][7] * v_matrix[7][o_token_index])
    + (score_matrix[i_token_length-1][8] * v_matrix[8][o_token_index])
    + (score_matrix[i_token_length-1][9] * v_matrix[9][o_token_index])
    + (score_matrix[i_token_length-1][10] * v_matrix[10][o_token_index])
    + (score_matrix[i_token_length-1][11] * v_matrix[11][o_token_index])
    + (score_matrix[i_token_length-1][12] * v_matrix[12][o_token_index])
    + (score_matrix[i_token_length-1][13] * v_matrix[13][o_token_index])
    + (score_matrix[i_token_length-1][14] * v_matrix[14][o_token_index])
    + (score_matrix[i_token_length-1][15] * v_matrix[15][o_token_index])
    + (score_matrix[i_token_length-1][16] * v_matrix[16][o_token_index])
    + (score_matrix[i_token_length-1][17] * v_matrix[17][o_token_index])
    + (score_matrix[i_token_length-1][18] * v_matrix[18][o_token_index])
    + (score_matrix[i_token_length-1][19] * v_matrix[19][o_token_index])
    + (score_matrix[i_token_length-1][20] * v_matrix[20][o_token_index])
    + (score_matrix[i_token_length-1][21] * v_matrix[21][o_token_index])
    + (score_matrix[i_token_length-1][22] * v_matrix[22][o_token_index])
    + (score_matrix[i_token_length-1][23] * v_matrix[23][o_token_index])
    + (score_matrix[i_token_length-1][24] * v_matrix[24][o_token_index])
    + (score_matrix[i_token_length-1][25] * v_matrix[25][o_token_index])
    + (score_matrix[i_token_length-1][26] * v_matrix[26][o_token_index])
    + (score_matrix[i_token_length-1][27] * v_matrix[27][o_token_index])
    + (score_matrix[i_token_length-1][28] * v_matrix[28][o_token_index])
    + (score_matrix[i_token_length-1][29] * v_matrix[29][o_token_index])
    + (score_matrix[i_token_length-1][30] * v_matrix[30][o_token_index])
    + (score_matrix[i_token_length-1][31] * v_matrix[31][o_token_index]);
end
```

在計算 o_token 當中我捨去用腦袋思考，直接用 case 分開 i_token_length 並直接爆開來，雖然過程是滿爽的，但是一想到那 critical path 之長就不經汗顏。

